Introduction to Deep Learning - Ex 1

By: Mohammad Awawda

Programming Task

Sub-section (a):

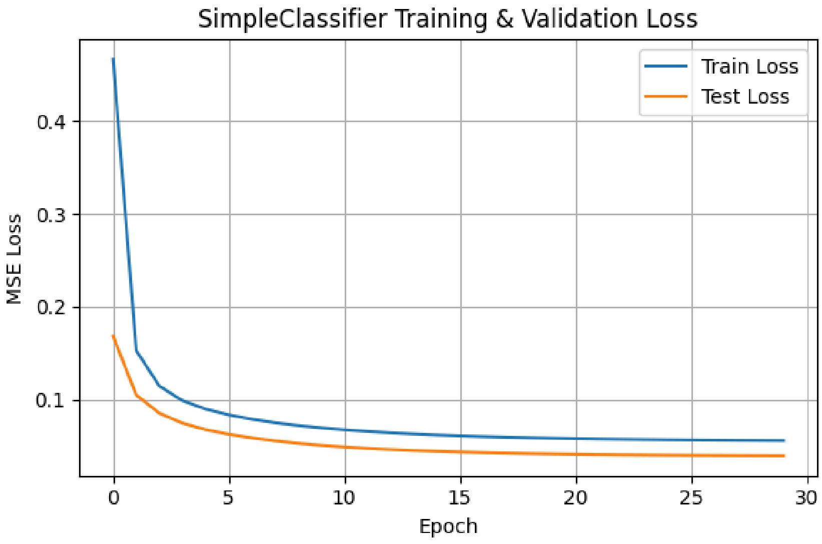
We chose to represent each peptide (string of 9 unique letters each corresponding to an amino acid) as a vector of integers, so that at index i the integer is the number of the i-th amino acid in the peptide. We kept a dictionary where each amino acid is matched to a unique integer between 0 and 19, which lacks any biological or chemical meaning. We chose this method since it isn't expensive in memory and preserves all of the information of the peptide - the location of each amino acid and its sequential appearance.

Sub-section (b):

The network is given the representation of a peptide as input, which we defined as a 9 integer long vector. We built an MLP of 2 inner layers, of the same dimension as the input layer and an output vector of boolean values of size 6. We chose to use MSE as our loss function, since it is a common method to measure distance between two vectors, such as our output representation.

We found that our chosen architecture resulted in a rapid convergence to a local minimum, which then flattens to a stable value. This may be caused by the lack of complexity in our model, which fails to capture the complexity of the physical and chemical properties which underlie interactions between the amino acids and the alleles they may or may not bind.

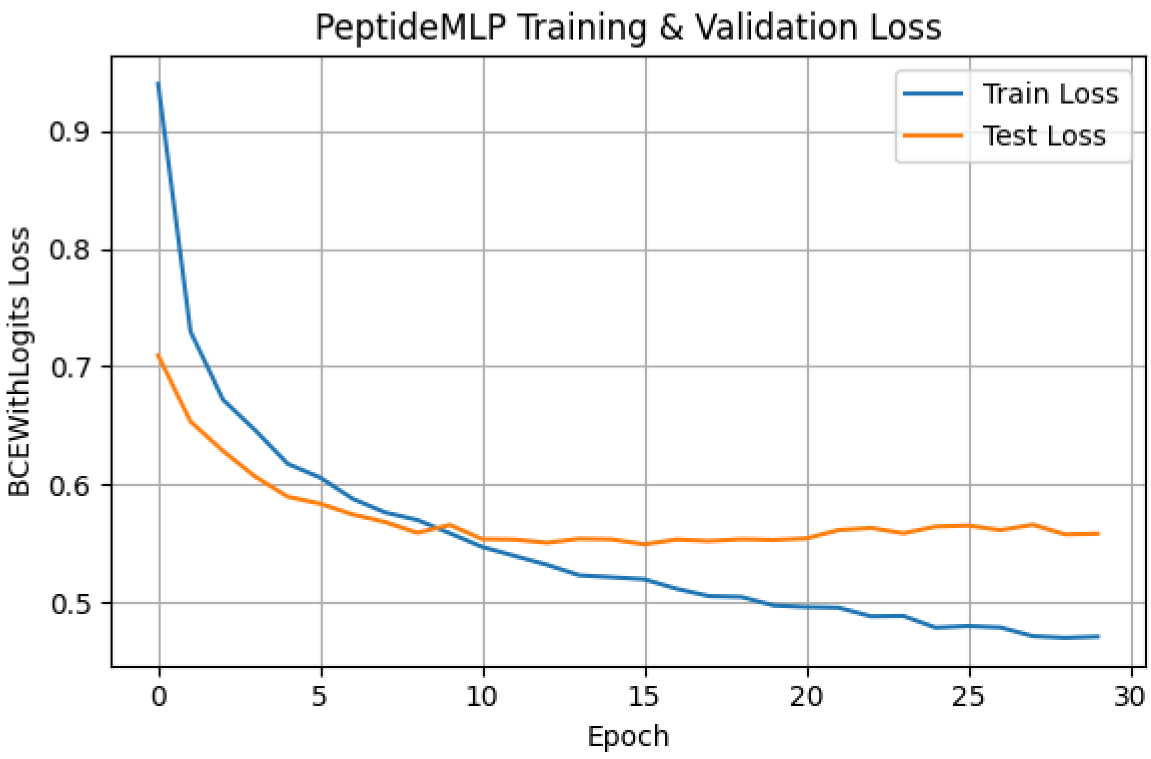
We suggest that since our architecture wasn't expressive enough, our model found a simple rule to quickly identify the majority, rather than finding meaningful patterns in the data. Furthermore, our loss function may not be ideal for a classification problem, and may also cause our model to converge quickly but insufficiently. In the next subsection, we use – which handles classification problems better than MSE, and also takes into account different proportions of the dataset, which we observed here.



Results (b): SimpleClassifier Training & Validation Loss
Epoch 00 — train_loss: 0.4665, test_loss: 0.1683
Epoch 01 — train_loss: 0.1530, test_loss: 0.1051
Epoch 02 — train_loss: 0.1146, test_loss: 0.0854
Epoch 03 — train_loss: 0.0990, test_loss: 0.0747
Epoch 04 — train_loss: 0.0899, test_loss: 0.0678
Epoch 05 — train_loss: 0.0837, test_loss: 0.0628
Epoch 06 — train_loss: 0.0790, test_loss: 0.0589
Epoch 07 — train_loss: 0.0752, test_loss: 0.0557
Epoch 08 — train_loss: 0.0722, test_loss: 0.0530
Epoch 09 — train_loss: 0.0697, test_loss: 0.0508
Epoch 10 — train_loss: 0.0676, test_loss: 0.0491
Epoch 11 — train_loss: 0.0659, test_loss: 0.0476
Epoch 12 — train_loss: 0.0644, test_loss: 0.0463
Epoch 13 — train_loss: 0.0631, test_loss: 0.0453
Epoch 14 — train_loss: 0.0620, test_loss: 0.0444
Epoch 15 — train_loss: 0.0611, test_loss: 0.0436
Epoch 16 — train_loss: 0.0603, test_loss: 0.0430
Epoch 17 — train_loss: 0.0596, test_loss: 0.0424
Epoch 18 — train_loss: 0.0590, test_loss: 0.0419
Epoch 19 — train_loss: 0.0585, test_loss: 0.0415
Epoch 20 — train_loss: 0.0581, test_loss: 0.0412
Epoch 21 — train_loss: 0.0577, test_loss: 0.0409
Epoch 22 — train_loss: 0.0574, test_loss: 0.0406
Epoch 23 — train_loss: 0.0571, test_loss: 0.0404
Epoch 24 — train_loss: 0.0568, test_loss: 0.0402
Epoch 25 — train_loss: 0.0566, test_loss: 0.0401
Epoch 26 — train_loss: 0.0564, test_loss: 0.0400
Epoch 27 — train_loss: 0.0563, test_loss: 0.0398
Epoch 28 — train_loss: 0.0561, test_loss: 0.0398
Epoch 29 — train_loss: 0.0560, test_loss: 0.0397

Sub-section (c):
We implemented the following changes to our model, and reached better results.
Network Architecture:
- **Input:** Nine amino-acid indices (shape = 9), each in {0,…,19},{0,...,19}{0,…,19}.
- **Embedding:** A learnable embedding layer maps each index to an 8-dim vector, producing a (9×8=72)-dim representation.
- **Hidden Layers:**
  - **Linear(72→128) → ReLU → Dropout=0.3**
  - **Linear(128→64) → ReLU → Dropout=0.3**
- **Output Layer: Linear(64→6)**, yielding six raw logits.
- **Loss & Optimization:**

    - **Loss:** BCEWithLogitsLoss with per-allele
    - **Optimizer:** Adam, with learning rate 0.001 and weight_decay=0.00001
- **Regularization:** Dropout in both hidden layers to reduce over-fitting on the limited positive examples.

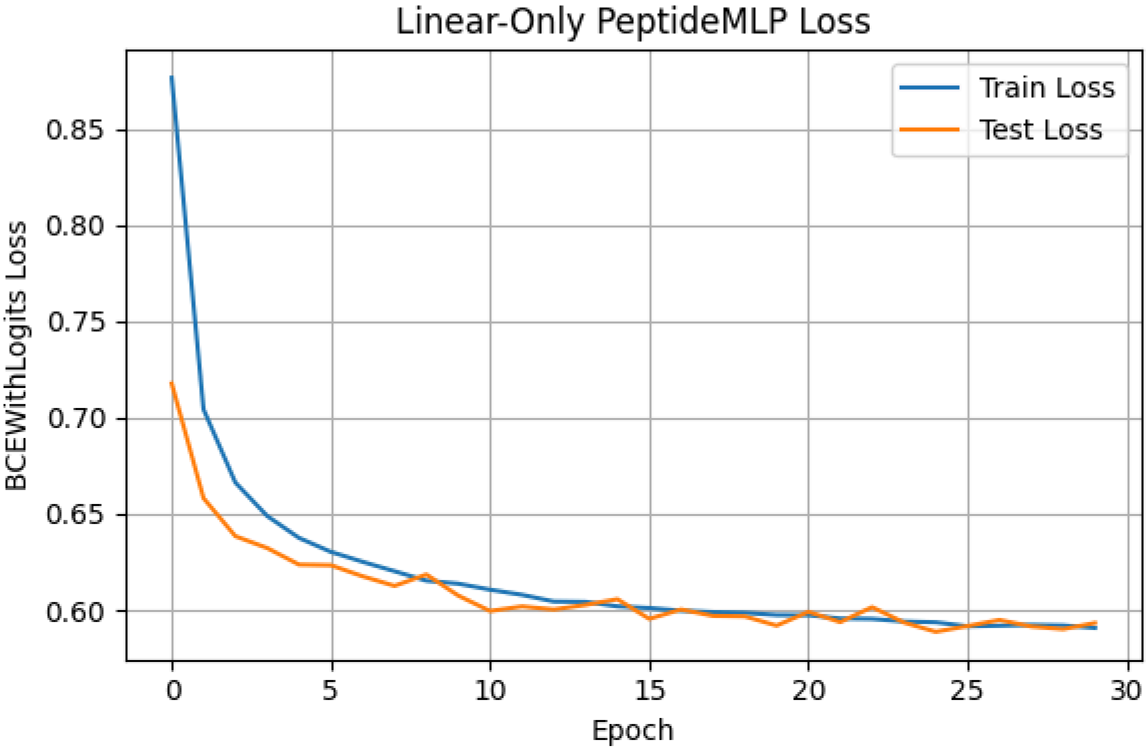


Results: PeptideMLP Training & Validation Loss

```
Epoch 01 — train_loss: 0.9404, test_loss:  0.7095
Epoch 02 — train_loss: 0.7296, test_loss:  0.6536
Epoch 03 — train_loss: 0.6722, test_loss:  0.6286
Epoch 04 — train_loss: 0.6460, test_loss:  0.6066
Epoch 05 — train_loss: 0.6175, test_loss:  0.5896
Epoch 06 — train_loss: 0.6059, test_loss:  0.5837
Epoch 07 — train_loss: 0.5879, test_loss:  0.5746
Epoch 08 — train_loss: 0.5763, test_loss:  0.5681
Epoch 09 — train_loss: 0.5698, test_loss:  0.5589
Epoch 10 — train_loss: 0.5585, test_loss:  0.5656
Epoch 11 — train_loss: 0.5468, test_loss:  0.5536
Epoch 12 — train_loss: 0.5392, test_loss:  0.5531
Epoch 13 — train_loss: 0.5317, test_loss:  0.5504
Epoch 14 — train_loss: 0.5227, test_loss:  0.5538
Epoch 15 — train_loss: 0.5211, test_loss:  0.5533
Epoch 16 — train_loss: 0.5194, test_loss:  0.5491
Epoch 17 — train_loss: 0.5113, test_loss:  0.5532
Epoch 18 — train_loss: 0.5051, test_loss:  0.5519
Epoch 19 — train_loss: 0.5044, test_loss:  0.5534
Epoch 20 — train_loss: 0.4971, test_loss:  0.5528
Epoch 21 — train_loss: 0.4958, test_loss:  0.5541
Epoch 22 — train_loss: 0.4952, test_loss:  0.5612
Epoch 23 — train_loss: 0.4878, test_loss:  0.5631
Epoch 24 — train_loss: 0.4882, test_loss:  0.5585
Epoch 25 — train_loss: 0.4782, test_loss:  0.5642
Epoch 26 — train_loss: 0.4796, test_loss:  0.5651
Epoch 27 — train_loss: 0.4784, test_loss:  0.5612
Epoch 28 — train_loss: 0.4710, test_loss:  0.5658
Epoch 29 — train_loss: 0.4696, test_loss:  0.5575
Epoch 30 — train_loss: 0.4705, test_loss:  0.5581
```

Sub-section (d):

After removing the non-linear operators from our network we noticed that the loss over the train and test sets converges at a much similar rate. The final results we obtain at epoch 30 without non-linear operators are significantly worse for the train set: 0.4705 with and 0.5907 without, but only slightly worse for the test set: 0.5581 with and 0.5933 without. We think that the non-linear operators increased the expressivity of our network, which resulted in overfitting to our train set.


Linear-Only PeptideMLP Loss

Results (d): Linear-Only PeptideMLP Loss

```
Epoch 01 — train_loss: 0.8764, test_loss:  0.7178
Epoch 02 — train_loss: 0.7043, test_loss:  0.6582
Epoch 03 — train_loss: 0.6663, test_loss:  0.6384
Epoch 04 — train_loss: 0.6489, test_loss:  0.6322
Epoch 05 — train_loss: 0.6375, test_loss:  0.6236
Epoch 06 — train_loss: 0.6303, test_loss:  0.6233
Epoch 07 — train_loss: 0.6251, test_loss:  0.6175
Epoch 08 — train_loss: 0.6202, test_loss:  0.6126
Epoch 09 — train_loss: 0.6153, test_loss:  0.6185
Epoch 10 — train_loss: 0.6137, test_loss:  0.6075
Epoch 11 — train_loss: 0.6106, test_loss:  0.5995
Epoch 12 — train_loss: 0.6079, test_loss:  0.6019
Epoch 13 — train_loss: 0.6045, test_loss:  0.6003
Epoch 14 — train_loss: 0.6043, test_loss:  0.6027
Epoch 15 — train_loss: 0.6019, test_loss:  0.6057
Epoch 16 — train_loss: 0.6011, test_loss:  0.5954
Epoch 17 — train_loss: 0.5997, test_loss:  0.6003
Epoch 18 — train_loss: 0.5990, test_loss:  0.5970
Epoch 19 — train_loss: 0.5986, test_loss:  0.5967
Epoch 20 — train_loss: 0.5975, test_loss:  0.5919
Epoch 21 — train_loss: 0.5974, test_loss:  0.5991
Epoch 22 — train_loss: 0.5956, test_loss:  0.5938
Epoch 23 — train_loss: 0.5954, test_loss:  0.6015
Epoch 24 — train_loss: 0.5941, test_loss:  0.5936
Epoch 25 — train_loss: 0.5937, test_loss:  0.5887
Epoch 26 — train_loss: 0.5916, test_loss:  0.5917
Epoch 27 — train_loss: 0.5919, test_loss:  0.5949
Epoch 28 — train_loss: 0.5923, test_loss:  0.5915
Epoch 29 — train_loss: 0.5920, test_loss:  0.5901
Epoch 30 — train_loss: 0.5907, test_loss:  0.5933
```

Sub-section (e):

```
Top 3 Spike 9-mers and alleles:
  Peptide: QPELDSFKE  Allele: A0101  Probability: 0.9993
  Peptide: STEIYQAGS  Allele: A0101  Probability: 0.9970
  Peptide: YYPDKVFRS  Allele: A2402  Probability: 0.9961
```

By: Awawda Mohammad

מבוא ללמידה עמוקה - תרגיל 1 - חלק תיאורטי.

שאלה 1

* יהיו $f:\mathbb{R}^d\to\mathbb{R}^m$, $g:\mathbb{R}^m\to\mathbb{R}^n$ העתקות לינאריות שמוגדרות לפי $f(x)=Ax$, $g(x)=Bx$, $A\in\mathbb{R}^{d\times m}$, $B\in\mathbb{R}^{m\times n}$

אז: $(f\circ g)(x)=f(g(x))=ABx$

אם כן, $h:\mathbb{R}^d\to\mathbb{R}^n$ שמוגדרת לפי $h(x)=(AB)x$ הינה גם היא טרנספורמציה לינארית.

* יהיו $f:\mathbb{R}^d\to\mathbb{R}^m$, $g:\mathbb{R}^m\to\mathbb{R}^n$ העתקות אפיניות, כלומר:

$$\forall k_1\in\mathbb{N}\quad a_1,\dots,a_{k_1}\in\mathbb{R}^d\quad \lambda_1,\dots,\lambda_{k_1}\in\mathbb{R}\quad \sum_{i=1}^{k_1}\lambda_i=1\Rightarrow f\Big(\sum_{i=1}^{k_1}\lambda_i a_i\Big)=\sum_{i=1}^{k_1}\lambda_i f(a_i)$$

$$\forall k_2\in\mathbb{N}\quad b_1,\dots,b_{k_2}\in\mathbb{R}^m\quad \sigma_1,\dots,\sigma_{k_2}\in\mathbb{R}\quad \sum_{i=1}^{k_2}\sigma_i=1\Rightarrow g\Big(\sum_{i=1}^{k_2}\sigma_i b_i\Big)=\sum_{i=1}^{k_2}\sigma_i g(b_i)$$

יהיו $k\in\mathbb{N}$, $a_1,\dots,a_k\in\mathbb{R}^d$. $\lambda_1,\dots,\lambda_k\in\mathbb{R}$ כך ש- $\sum_{i=1}^{k}\lambda_i=1$ נתבונן בהרכבה של g על f:

$$(g\circ f)\Big(\sum_{i=1}^{k}\lambda_i a_i\Big)=g\Big(f\Big(\sum_{i=1}^{k}\lambda_i a_i\Big)\Big)\underset{\uparrow\ \text{f אפינית}}{=}g\Big(\sum_{i=1}^{k}\lambda_i f(a_i)\Big)\underset{\uparrow\ \text{g אפינית}}{=}\sum_{i=1}^{k}\lambda_i\, g(f(a_i))=\sum_{i=1}^{k}\lambda_i (g\circ f)(a_i)$$

ומכאן שגם $g\circ f$ אפינית.

שאלה 2

א) מהנתון פונק' $f:\mathbb{R}^2\to\mathbb{R}$ גזירה וקמורה, נרצה למצוא את נק' המינימום (x_{min},y_{min}).

נתחל מנקודה אקראית $(x^{(1)},y^{(1)})$ ונתקדם בצעדים לפי $\eta>0$ (קצב הלמידה) בכיוון הירידה של הפונקציה - כלומר בניגוד לגרדיאנט בנקודה $(x^{(t)},y^{(t)})$. אז משוואת העדכון: $(x^{(t+1)},y^{(t+1)})=(x^{(t)},y^{(t)})-\eta\nabla f(x^{(t)},y^{(t)})$

נחשב את הגרדיאנט של $f(x,y)=10(x-1)^2+(y+1)^2/10$:

$$\nabla f(x,y)=\begin{bmatrix}\frac{\partial f}{\partial x}\\ \frac{\partial f}{\partial y}\end{bmatrix}=\begin{bmatrix}20(x-1)\\ \frac{1}{5}(y+1)\end{bmatrix}$$

מכאן נקבל את כלל העדכון הבא:

$$x^{(t+1)}=x^{(t)}-20\eta(x^{(t)}-1)=(1-20\eta)x^{(t)}+20\eta$$

$$y^{(t+1)}=y^{(t)}-\frac{\eta}{5}(y^{(t)}+1)=\Big(1-\frac{\eta}{5}\Big)y^{(t)}-\frac{\eta}{5}$$

ב) מהסעיף הקודם, קיבלנו: $x^{(t+1)}=A_x\cdot x^{(t)}+B_x$, $y^{(t+1)}=A_y\cdot y^{(t)}+B_y$

נשים לב שעבור $t\to\infty$ נרצה שהסדרה תתכנס לנקודת מינימום בפרט ערך קבוע (x_{min},y_{min}) ולכן,

$$\lim_{t\to\infty}x^{(t)}=x_{min}\iff\lim_{t\to\infty}\Big[A_x^t\cdot x_0+\sum_{k=0}^{t-1}A_x^k B_x\Big]=x_{min}\iff\lim_{t\to\infty}A_x^t\Big(x_0+B\sum_{k=1}^{t}\frac{1}{A_x^k}\Big)\in\mathbb{R}$$

באופן רקורסיבי מתוך משוואת $x^{(t+1)}$

תנאי זה מתקיים אמ"מ $|A_x|<1$ (אילו $A_x=1$ נקבל $\lim_{t\to\infty}x^{(t)}=\lim_{t\to\infty}(x_0+tB)\to\infty$)

ולכן סה"כ נקבל $|A_y|<1$, $|A_x|<1$. מסעיף 1:

$$|1-20\eta|<1\Rightarrow 0<\eta<0.1\quad,\quad \Big|1-\frac{\eta}{5}\Big|<1\Rightarrow 0<\eta<10$$

החסם ההדוק יותר מתקבל עבור x ולכן $0<\eta<0.1$.

* אילו נבחר $0.1\le\eta<10$ אז $x^{(t)}$ יתבדר (ישאף ל-∞).

* אילו נבחר $\eta>10$ או $\eta<0$ אז $x^{(t)},y^{(t)}$ שניהם יתבדרו, ולא נמצא מינימום גלובלי.

שאלה ב - המשך:

ג) נחשב את ההסיאן של הפונקציה:

$$H_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial y \partial x} \\ \frac{\partial^2 f}{\partial x \partial y} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix} = \begin{bmatrix} 20 & 0 \\ 0 & 1/5 \end{bmatrix}$$

הע״ע של ההסיאן מגדירים מי מבין הירידות/עליות תלולות יותר.

במקרה הזה קיבלנו $\frac{\partial^2 f}{\partial x^2} = 20 > 1/5 = \frac{\partial^2 f}{\partial y^2}$ לכן x הינו הגורם שקובע את קצב הלמידה המקסימלי $\eta \le \frac{2}{20} = 0.1$

ד) באותו אופן נקבל שהמשתנה y מתכנס לאט יותר, כי הע״ע שלו בהסיאן הוא הקטן יותר.

$y^{(t+1)} = (1 - \frac{\eta}{5}) y^{(t)} - \frac{\eta}{5}$ כך שעבור $\eta < 0.1$ y מתקרב יותר מהר ל-1 מאשר x.

שאלה 3: בהינתן θ, ו- $\hat{\theta}$ הערך החזוי

$$\Delta = \min(|\hat{\theta} - \theta|, 360° - |\hat{\theta} - \theta|)$$

דרך Δ לוקחת את הזווית המינימלית מבין ה2.

$$L(\hat{\theta}, \theta) = \Delta^2 = \min(|\hat{\theta} - \theta|, 360 - |\hat{\theta} - \theta|)^2$$

והפסודו קוד הבא:

```
def Circular_loss (preds, targets):

    total=0
      N = len(preds)

    for i=1, .., N
        'Compute $\Delta^2$
        total = total + $\Delta$

    return $\frac{total}{N}$
```

שאלה 4

(a) $\frac{\partial}{\partial x} f(x+y,2x,z)$

נגדיר את הפונקציה $g: \mathbb{R} \to \mathbb{R}^3$ באופן הבא: $g(x) = \begin{pmatrix} x+y \\ 2x \\ z \end{pmatrix}$

אז מתקיים $f\begin{pmatrix} x+y \\ 2x \\ z \end{pmatrix} = f\left(g\begin{pmatrix} x \\ y \\ z \end{pmatrix}\right)$ וכעת ניתן להפעיל את כלל השרשרת: $D_x(f \circ g) = D_{g(x)}(f) \cdot D_x(g)$

מתקיים:

$$(Dg)_x = \begin{pmatrix} \frac{\partial x+y}{\partial x} \\ \frac{\partial 2x}{\partial x} \\ \frac{\partial z}{\partial x} \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 0 \end{pmatrix}$$

ולכן נקבל:

$$\frac{\partial f(x+y,2x,z)}{\partial x} = \frac{\partial f(g(x))}{\partial x} = \frac{\partial f}{\partial g(x)} \cdot \frac{\partial g}{\partial x} = \frac{\partial f}{\partial g(x)} \cdot \begin{pmatrix} 1 \\ 2 \\ 0 \end{pmatrix}$$

(b) $f_1\Big(f_2(\ldots f_n(x))\Big)$

$$\frac{\partial (f_1 \circ f_2 \circ \cdots \circ f_n)(x)}{\partial x} = \frac{\partial f_1}{\partial f_2(f_3 \circ \cdots \circ f_n(x))} \cdot \frac{\partial f_2}{\partial f_3(f_4 \circ \cdots f_n(x))} \cdot \ldots \cdot \frac{\partial f_{n-1}}{\partial f_n(x)} \cdot \frac{\partial f_n}{\partial x}$$

(c) $f_1\Big(x, f_2\big(x, f_3(\ldots f_{n-1}(x, f_n(x)))\big)\Big)$

$$\frac{\partial f_1(x, f_2(x, \ldots f_{n-1}(x, f_n(x))))}{\partial x} \underset{\text{כלל השרשרת}}{=} \frac{\partial f_1}{\partial [x, f_2(x, .. f_{n-1}(x, f_n(x)))]} \cdot \frac{\partial [x, f_2(x, .. f_{n-1}(x, f_n(x)))]}{\partial x} =$$

$$= \left[\frac{\partial f_1}{\partial x}, \frac{\partial f_1}{\partial f_2(x, .. f_{n-1}(x, f_n(x)))}\right] \cdot \begin{bmatrix} \frac{\partial x}{\partial x} \\ \frac{\partial f_2(x, .. f_{n-1}(x, f_n(x)))}{\partial x} \end{bmatrix} = \frac{\partial f_1}{\partial x} \cdot 1 + \frac{\partial f_1}{\partial f_2(x, .. f_{n-1}(x, f_n(x)))} \cdot \overset{\circledast}{\frac{\partial f_2(x, .. f_{n-1}(x, f_n(x)))}{\partial x}} =$$

↑ פירוק ⊛ באופן רקורסיבי.

$$= \frac{\partial f_1}{\partial x} + \frac{\partial f_1}{\partial f_2(x, .. f_{n-1}(x, f_n(x)))} \left(\frac{\partial f_2}{\partial x} + \frac{\partial f_2}{\partial f_3(x, \ldots f_{n-1}(x, f_n(x)))} \left(\ldots + \frac{\partial f_{n-2}}{\partial f_{n-1}(x, f_n(x))} \left(\frac{\partial f_{n-1}}{\partial x} + \frac{\partial f_{n-1}}{\partial f_n(x)} \frac{\partial f_n}{\partial x} \right)\right)\right)$$

(d) $f\Big(x + g(x + h(x))\Big)$

$$\frac{\partial f(x+g(x+h(x)))}{\partial x} = \frac{\partial f}{\partial (x+g(x+h(x)))} \cdot \frac{\partial x + g(x+h(x))}{\partial x} =$$

$$= \frac{\partial f}{\partial (x+g(x+h(x)))} \cdot \left(1 + \frac{\partial g}{\partial x+h(x)} \cdot \frac{\partial x+h(x)}{\partial x}\right) =$$

$$= \frac{\partial f}{\partial (x+g(x+h(x)))} \cdot \left(1 + \frac{\partial g}{\partial x+h(x)} \cdot \left(1 + \frac{\partial h}{\partial x}\right)\right)$$